

Spring REST HandsOn

WEEK-3:

1.spring-rest-handson

Create a Spring Web Project using Maven

```
> springlearnrest [boot] [devtools] [microservice main]
> src/main/java
> com.springlearnrest
> SpringlearnrestApplication.java
> src/main/resources
> src/test/java
JRE System Library [JavaSE-17]
Maven Dependencies
> src
target
HELP.md
mvnw
mvnw.cmd

package com.springlearnrest;

import org.springframework.boot.SpringApplication;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class SpringlearnrestApplication {

    private static final Logger LOGGER =
        LoggerFactory.getLogger(SpringlearnrestApplication.class);

    public static void main(String[] args) {

        LOGGER.info("Application Started");

        SpringApplication.run(SpringlearnrestApplication.class, args);

        LOGGER.info("Application Running");
    }
}
```

```

Application      : Starting SpringlearnrestApplication using Java 21.0.9 with PID 241
Application      : No active profile set, falling back to 1 default profile: "default"
PropertySources : Devtools property defaults active! Set 'spring.devtools.add-property-value' to
PropertySources : For additional web related logging consider setting the 'logging.level.org.springframework
WebServer        : Tomcat initialized with port 8080 (http)
StandardService : Starting service [Tomcat]
StandardEngine   : Starting Servlet engine: [Apache Tomcat/11.0.22]
ContextInitializer : Root WebApplicationContext: initialization completed in 512 ms
WebServer        : Tomcat started on port 8080 (http) with context path '/'
Application      : Started SpringlearnrestApplication in 1.036 seconds (process running for 1.041)
Application      : Application Running

```

```

<parent>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-parent</artifactId>
  <version>4.1.0</version>
  <relativePath/> <!-- lookup parent from repository -->
</parent>
<groupId>com.springlearnrest</groupId>
<artifactId>events</artifactId>
<version>3.2.5</version>
<name>springlearnrest</name>
<description/>
<url/>
<licenses>
  <license/>
</licenses>
<developers>
  <developer/>
</developers>
<scm>
  <connection/>
  <developerConnection/>
  <tag/>
  <url/>
</scm>
<properties>
  <java.version>17</java.version>
</properties>
<dependencies> Add Spring Boot Starters...
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-webmvc</artifactId>
  </dependency>

  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-devtools</artifactId>
    <scope>runtime</scope>
    <optional>true</optional>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-webmvc-test</artifactId>
    <scope>test</scope>
  </dependency>
</dependencies>

<build>
  <plugins>
    <plugin>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-maven-plugin</artifactId>
    </plugin>
  </plugins>
</build>
</project>

```

Hands-on 4: Spring Core – Load Country from Spring Configuration XML

```
2026-07-12T17:19:59.497+05:30 INFO 5624 --- [spri
2026-07-12T17:19:59.498+05:30 INFO 5624 --- [spri
2026-07-12T17:20:00.122+05:30 INFO 5624 --- [spri
2026-07-12T17:20:00.136+05:30 INFO 5624 --- [spri
2026-07-12T17:20:00.137+05:30 INFO 5624 --- [spri
2026-07-12T17:20:00.165+05:30 INFO 5624 --- [spri
2026-07-12T17:20:00.458+05:30 INFO 5624 --- [spri
2026-07-12T17:20:00.463+05:30 INFO 5624 --- [spri
Inside Country Constructor
Country{code='IN', name='India'}
```

```
package com.cognizant.springrestlearn;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class SpringrestlearnApplication {

    public static void main(String[] args)
    {
        SpringApplication.run(SpringrestlearnApplication.class, args);

        displayCountry();
    }
    public static void displayCountry() { 1 usage

        ApplicationContext context =
            new ClassPathXmlApplicationContext( configLocation: "country.xml");

        Country country =
            context.getBean( name: "country", Country.class);

        System.out.println(country);
    }
}
```

springrestlearn ~/Downloads/springrestlearn

> .idea

> .mvn

> .settings

✓ src

✓ main

✓ java

✓ com.cognizant.springrestlearn

Country

SpringrestlearnApplication

✓ resources

static

templates

application.properties

country.xml

```
package com.cognizant.springrestlearn;

public class Country { no usages

    private String code; 3 usages
    private String name; 3 usages

    public Country() { no usages
        System.out.println("Inside Country Constructor")
    }

    public String getCode() { no usages
        return code;
    }

    public void setCode(String code) { no usages
        this.code = code;
    }

    public String getName() { no usages
        return name;
    }

    public void setName(String name) { no usages
        this.name = name;
    }

    @Override
    public String toString() {
        return "Country{" +
            "code='" + code + '\'' +
            ", name='" + name + '\'' +
            '}';
    }
}
```

PDF-2:Hello World RESTful Web Service



localhost:8080/hello

Hello World!!

```
package com.cognizant.springrestlearn;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class SpringrestlearnApplication {

    public static void main(String[] args)
    {
        SpringApplication.run(SpringrestlearnApplication.class, args);
    }
}
```

```

package com.cognizant.springrestlearn.controller;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class HelloController {

    @GetMapping("/hello")
    public String sayHello() {
        return "Hello World!!";
    }
}

```

REST - Country Web Service:

POST ▼ | http://localhost:8080/country

☰ Docs
Params
Auth ●
Headers (9)
Body ●
Scripts
Se

raw ▼
JSON ▼
⚙️

```

1  {
2  |   "code": "IN",
3  |   "name": "India"
4  | }

```

↙

Body ▼
🔄
200 OK
• 200 ms
• 184

{{} JSON ▼
▶ Preview
🖼 Visualize ▼
⌵

```

1  {
2  |   "code": "IN",
3  |   "name": "India"
4  | }

```

```

package com.cognizant.springrestlearn.controller;

import com.cognizant.springrestlearn.Country;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class CountryController {

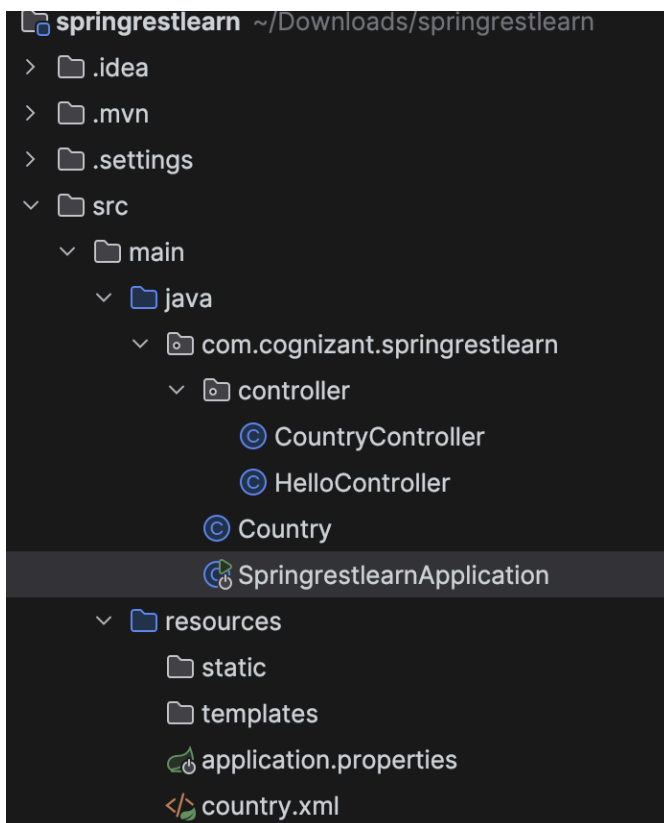
    @RequestMapping("/country")
    public Country getCountryIndia() {

        Country country = new Country();

        country.setCode("IN");
        country.setName("India");

        return country;
    }
}

```



REST – Get Country Based on Country Code

GET

▼

http://localhost:8080/countries/IN

Docs

Params

Auth ●

Headers (7)

Body

Scripts

Settings

raw ▼

JSON ▼

⚙

1

⌕ to Ask AI

Body ▼

🕒

200 OK

• 133 ms • 184

{ } JSON ▼

▶ Preview

🖼 Visualize ▼

⌕

1 {

2 |

3 | "code": "IN",

4 | "name": "India"

5 }

GET

▼

http://localhost:8080/countries/in

Docs

Params

Auth ●

Headers (7)

Body

Scripts

raw ▼

JSON ▼

1

⌕ to Ask AI

Body ▼

🕒

200 OK

• 6 ms

{ } JSON ▼

▶ Preview

🖼 Visualize ▼

⌕

1 {

2 |

3 | "code": "IN",

4 | "name": "India"

5 }


```

package com.cognizant.springrestlearn.controller;

import com.cognizant.springrestlearn.Country;
import com.cognizant.springrestlearn.service.CountryService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;

@RestController
public class CountryController {

    @Autowired
    private CountryService countryService;

    @RequestMapping("/country")
    public Country getCountryIndia() {
        return new Country( code: "IN", name: "India");
    }

    @GetMapping("/countries/{code}")
    public Country getCountry(@PathVariable String code) {

        return countryService.getCountry(code);
    }

}

```

```

package com.cognizant.springrestlearn.service;

import com.cognizant.springrestlearn.Country;
import org.springframework.stereotype.Service;

import java.util.ArrayList;
import java.util.List;

@Service
public class CountryService {

    private static final List<Country> countryList = new ArrayList<>();

    static {
        countryList.add(new Country( code: "IN", name: "India"));
        countryList.add(new Country( code: "US", name: "United States"));
        countryList.add(new Country( code: "JP", name: "Japan"));
        countryList.add(new Country( code: "DE", name: "Germany"));
    }

    public Country getCountry(String code) {

        for (Country country : countryList) {

            if (country.getCode().equalsIgnoreCase(code)) {
                return country;
            }

        }

        return null;
    }

}

```

PDF 3:

Employee & Department REST APIs

localhost:8080/employees

etty print ☐

```
"id":1,"name":"Rahul","salary":50000.0},{ "id":2,"name":"Priya","salary":60000.0}
```

```
package com.cognizant.springrestlearn;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class SpringrestlearnApplication {

    public static void main(String[] args)
    {
        SpringApplication.run(SpringrestlearnApplication.class, args);
    }

}
```

```
package com.cognizant.springrestlearn.service;

import com.cognizant.springrestlearn.dao.EmployeeDao;
import com.cognizant.springrestlearn.model.Employee;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;

import java.util.List;

@Service
public class EmployeeService {

    @Autowired
    EmployeeDao employeeDao;

    public List<Employee> getAllEmployees(){
        return employeeDao.getAllEmployees();
    }

}
```

```

package com.cognizant.springrestlearn.model;

public class Employee { 10 usages
    private int id; 3 usages
    private String name; 3 usages
    private double salary; 3 usages

    public Employee() { no usages
    }

    public Employee(int id, String name, double salary) { 3 usages
        this.id = id;
        this.name = name;
        this.salary = salary;
    }

    public int getId() { no usages
        return id;
    }

    public void setId(int id) { no usages
        this.id = id;
    }

    public String getName() { no usages
        return name;
    }

    public void setName(String name) { no usages
        this.name = name;
    }
}

```

```

package com.cognizant.springrestlearn.dao;

import com.cognizant.springrestlearn.model.Employee;
import org.springframework.stereotype.Repository;

import java.util.ArrayList;
import java.util.List;

@Repository
public class EmployeeDao {

    private static final List<Employee> EMPLOYEE_LIST = new ArrayList<>(); 4 usages

    static {

        EMPLOYEE_LIST.add(new Employee( id: 1, name: "Rahul", salary: 50000));
        EMPLOYEE_LIST.add(new Employee( id: 2, name: "Priya", salary: 60000));
        EMPLOYEE_LIST.add(new Employee( id: 3, name: "Amit", salary: 70000));
    }

    public List<Employee> getAllEmployees(){ 1 usage

        return EMPLOYEE_LIST;
    }
}

```

```
package com.cognizant.springrestlearn.controller;

import com.cognizant.springrestlearn.model.Employee;
import com.cognizant.springrestlearn.service.EmployeeService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

import java.util.List;

@RestController
public class EmployeeController {

    @Autowired
    EmployeeService employeeService;

    @GetMapping("/employees")
    public List<Employee> getAllEmployees(){

        return employeeService.getAllEmployees();
    }
}
```

PDF:5-

```

package com.cognizant.springrestlearn.security;

import io.jsonwebtoken.Jwts;
import io.jsonwebtoken.SignatureAlgorithm;
import io.jsonwebtoken.security.Keys;

import javax.crypto.SecretKey;
import java.util.Date;

public class JwtUtil { 3 usages

    private static final SecretKey SECRET_KEY = 1 usage
        Keys.secretKeyFor(SignatureAlgorithm.HS256);

    public String generateToken(String username) { 1 usage

        return Jwts.builder()
            .setSubject(username)
            .setIssuedAt(new Date())
            .setExpiration(new Date(System.currentTimeMillis() + 3600000))
            .signWith(SECRET_KEY)
            .compact();
    }
}

```



localhost:8080/authenticate

Pretty print ☐

```
{"token":"eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJ1c2VyIi
```

```

package com.cognizant.springrestlearn.security;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.config.Customizer;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.web.SecurityFilterChain;

@Configuration
public class SecurityConfig {

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http)
        throws Exception {

        http
            .csrf(CsrfConfigurer<HttpSecurity> csrf -> csrf.disable())
            .authorizeHttpRequests(AuthorizationManagerRequestMat... auth -> auth
                .requestMatchers("/authenticate").permitAll()
                .anyRequest().authenticated())
            .httpBasic(Customizer.withDefaults());

        return http.build();
    }
}

```

```

package com.cognizant.springrestlearn.model;

public class AuthenticationResponse { 3 usages

    private String token; 3 usages

    public AuthenticationResponse() { no usages
    }

    public AuthenticationResponse(String token) { this.token = token; }

    public String getToken() { return token; }

    public void setToken(String token) { no usages
        this.token = token;
    }
}

```